telecom-customer-churn-analysis

[59]:
```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[60]:
```python
df = pd.read_csv("customer churn.csv")
df.head()
```

[60]:
```
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  … DeviceProtection  \
0  No phone service             DSL             No  …               No
1                No             DSL            Yes  …              Yes
2                No             DSL            Yes  …               No
3  No phone service             DSL            Yes  …              Yes
4                No     Fiber optic             No  …               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
```

[61]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

[62]:
```
# replacing blanks with 0
df["TotalCharges"] = df["TotalCharges"].replace(" ","0")
```

[63]:
```
# change data type
df["TotalCharges"] = df["TotalCharges"].astype("float")
```

[64]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
```

```
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   float64
 20  Churn             7043 non-null   object
dtypes: float64(2), int64(2), object(17)
memory usage: 1.1+ MB
```

[65]:
```
df.isnull().sum()
```

[65]:
```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

[66]:
```
df.describe()
```

[66]:

	SeniorCitizen	tenure	MonthlyCharges	TotalCharges
count	7043.000000	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692	2279.734304
std	0.368612	24.559481	30.090047	2266.794470
min	0.000000	0.000000	18.250000	0.000000
25%	0.000000	9.000000	35.500000	398.550000
50%	0.000000	29.000000	70.350000	1394.550000
75%	0.000000	55.000000	89.850000	3786.600000
max	1.000000	72.000000	118.750000	8684.800000

[67]:

```
# check duplicate customer_id
df["customerID"].duplicated().sum()
```

[67]: np.int64(0)

[68]:

```
# convert 0 and 1 values of senior citizen to yes/no to make it easier to␣
↪understand
def conv(value):
    if value==1:
        return "yes"
    else:
        return "no"

df["SeniorCitizen"] = df["SeniorCitizen"].apply(conv)
```

[69]:

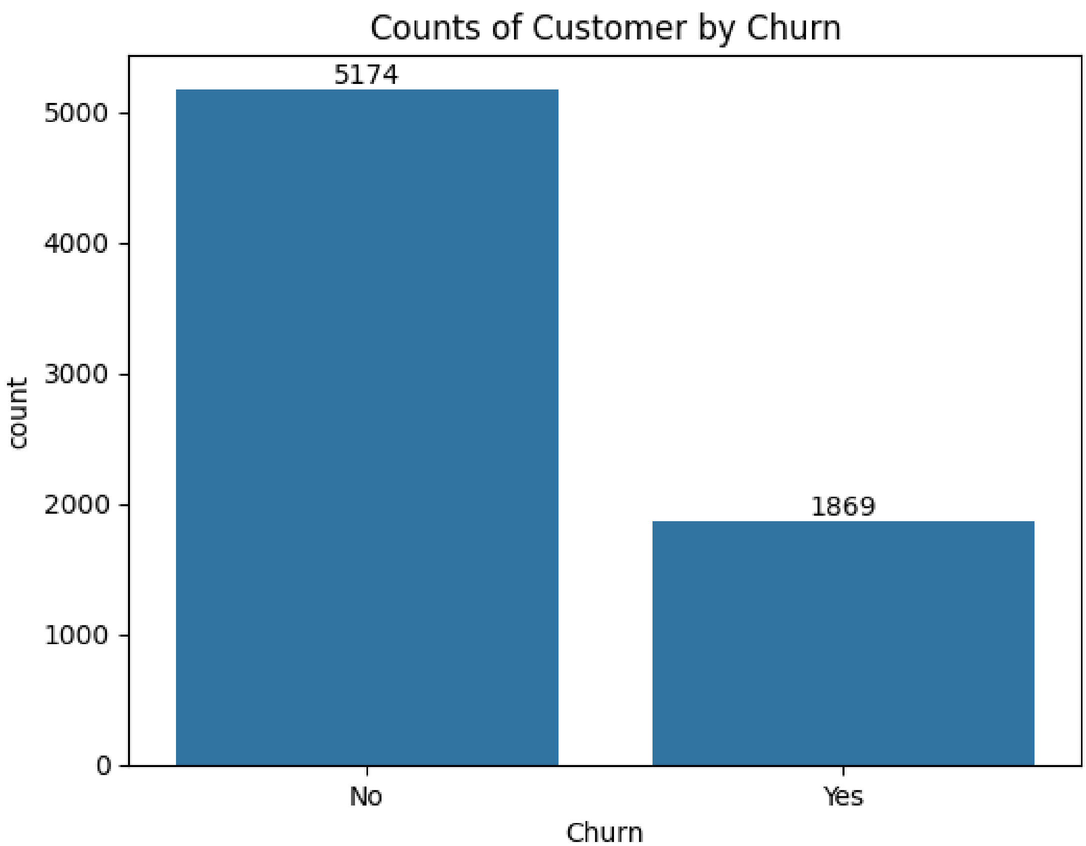
```
df.head()
```

[69]:

```
   customerID  gender SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female            no     Yes         No       1           No
1  5575-GNVDE    Male            no      No         No      34          Yes
2  3668-QPYBK    Male            no      No         No       2          Yes
3  7795-CFOCW    Male            no      No         No      45           No
4  9237-HQITU  Female            no      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  … DeviceProtection  \
0  No phone service             DSL             No  …               No
1                No             DSL            Yes  …              Yes
2                No             DSL            Yes  …               No
3  No phone service             DSL            Yes  …              Yes
4                No     Fiber optic             No  …               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes
```

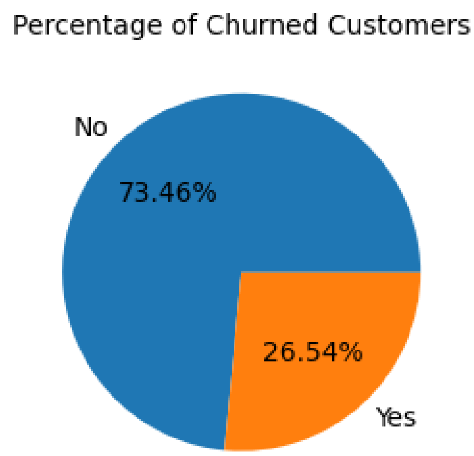
```
                PaymentMethod MonthlyCharges  TotalCharges  Churn
0            Electronic check          29.85         29.85     No
1                Mailed check          56.95       1889.50     No
2                Mailed check          53.85        108.15    Yes
3   Bank transfer (automatic)          42.30       1840.75     No
4            Electronic check          70.70        151.65    Yes

[5 rows x 21 columns]
```

[70]:
```
ax = sns.countplot(x="Churn", data=df)
ax.bar_label(ax.containers[0])
plt.title("Counts of Customer by Churn")
plt.show()
```



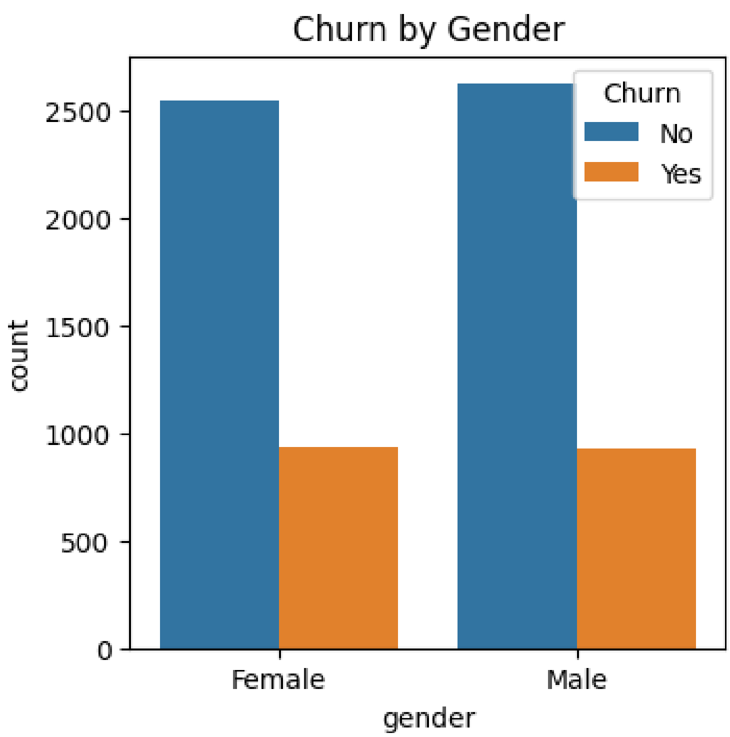
[71]:
```
plt.figure(figsize = (3,4))
gb = df.groupby("Churn").agg({"Churn":"count"})
plt.pie(gb["Churn"], labels=gb.index, autopct="%1.2f%%")                  #␣
 ↪labels me grup by ka index value
plt.title("Percentage of Churned Customers", fontsize=10)
```

```
plt.show()
```

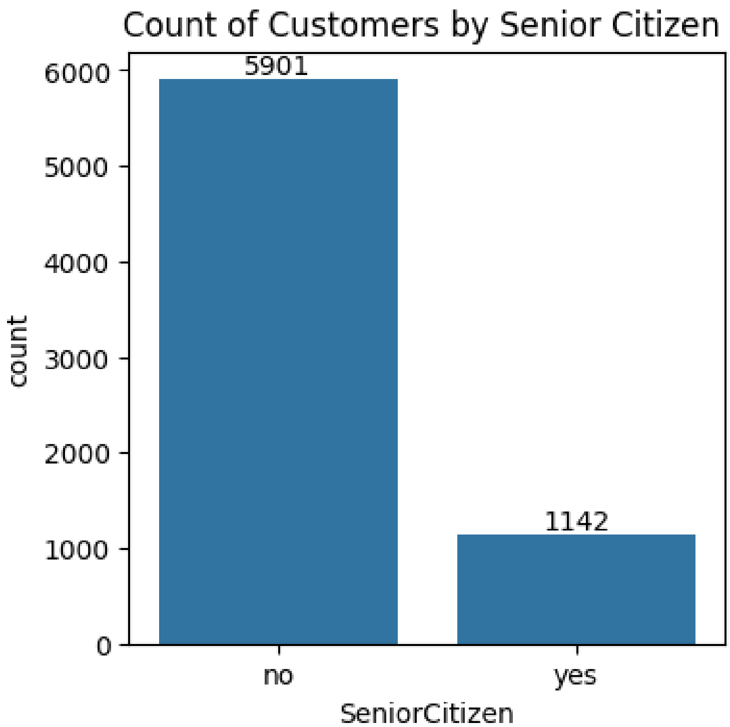


#for the given pie chart we can conclude that 26.54% of our customers have churned out. Now let's explore the reason behind it

[72]:
```
plt.figure(figsize=(4,4))
sns.countplot(x="gender", data=df, hue="Churn")
plt.title("Churn by Gender")
plt.show()
```

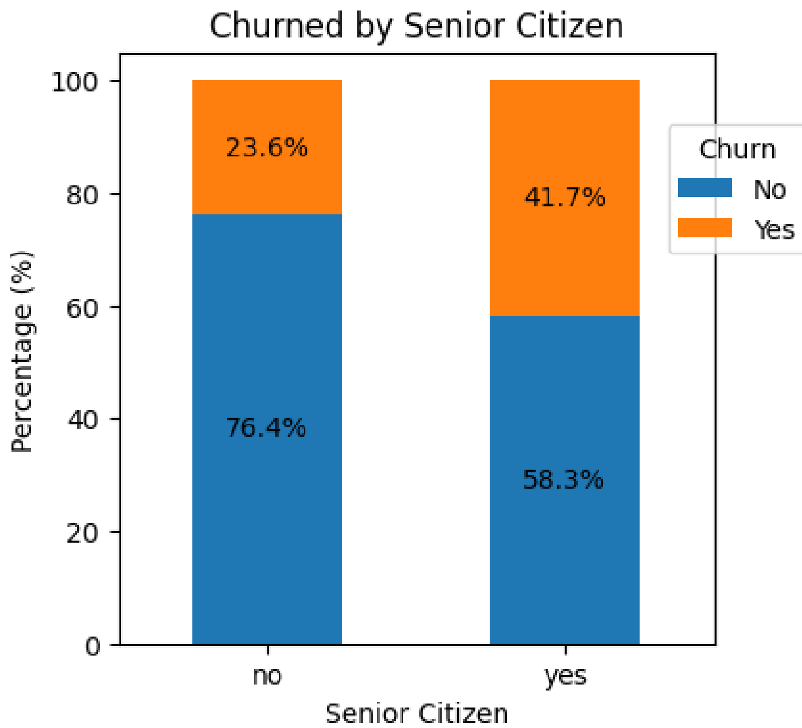


[73]:
```
plt.figure(figsize=(4,4))
ax = sns.countplot(x="SeniorCitizen", data=df)
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Senior Citizen ")
plt.show()
```



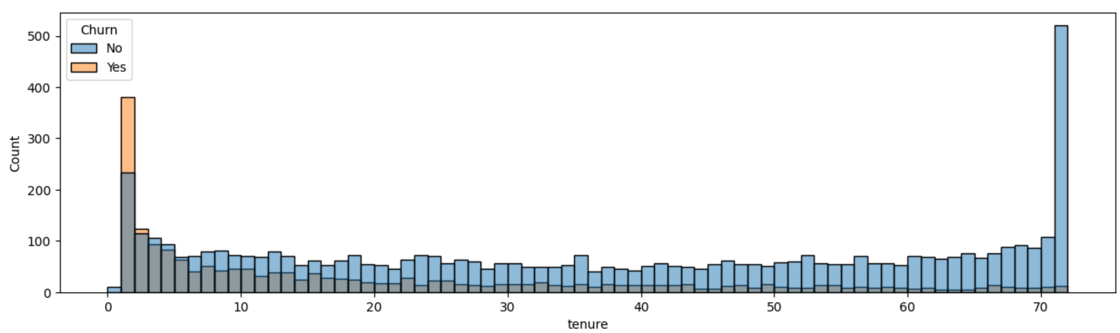
[74]:
```
# normalize use for proportion (means % part), use unstack for multilevel index␣
↪convert into column
total_counts = df.groupby("SeniorCitizen")["Churn"].
↪value_counts(normalize=True).unstack()*100

fig,ax = plt.subplots(figsize=(4,4))

total_counts.plot(kind="bar", stacked=True, ax=ax ,color=["#1f77b4", "#ff7f0e"])
# use stacked for multiple categories show in 1 bar chart

for p in ax.patches:
    width,height = p.get_width(),p.get_height()             # for getting bar␣
↪width and height
    x,y = p.get_xy()                                        # for getting bar␣
↪x and y location
    ax.text(x + width/2, y + height/2, f"{height:.1f}%", ha = "center",␣
↪va="center")  # horizontal and vertical alignment are center


plt.title("Churned by Senior Citizen")
plt.xlabel("Senior Citizen")
plt.ylabel("Percentage (%)")
plt.xticks(rotation=0)
```

```
plt.legend(title="Churn", bbox_to_anchor= (0.9,0.9))
plt.show()
```



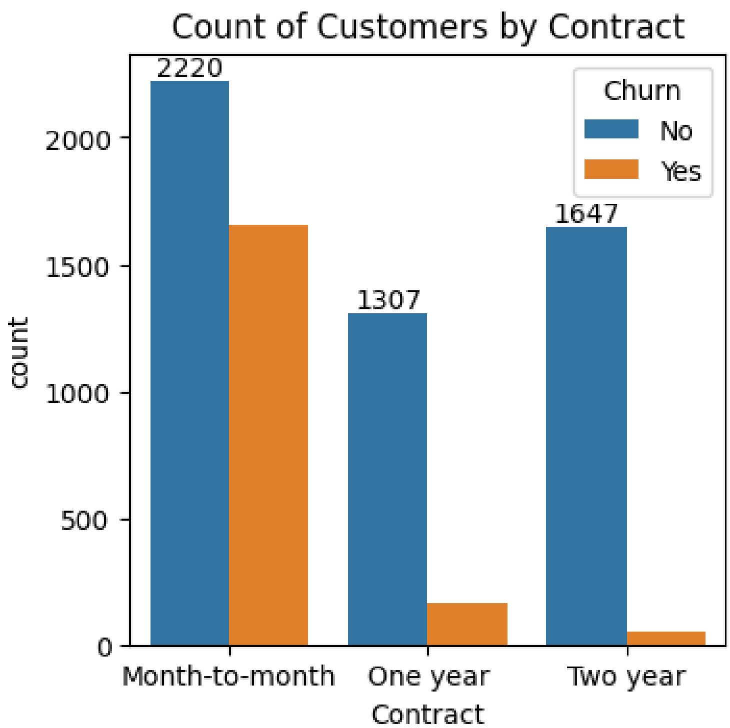
#comparative a greater percentage of people in senior citizen category have churned out

[75]:
```
plt.figure(figsize=(15,4))
sns.histplot(x = "tenure", data=df, bins=72, hue="Churn")
plt.show()
```



#people who have used our services for a long time have stayed and people who have used our serives 1 or 2 months have churned

[76]:
```
plt.figure(figsize=(4,4))
ax = sns.countplot(x="Contract", data=df, hue="Churn")
ax.bar_label(ax.containers[0])
plt.title("Count of Customers by Contract")
plt.show()
```



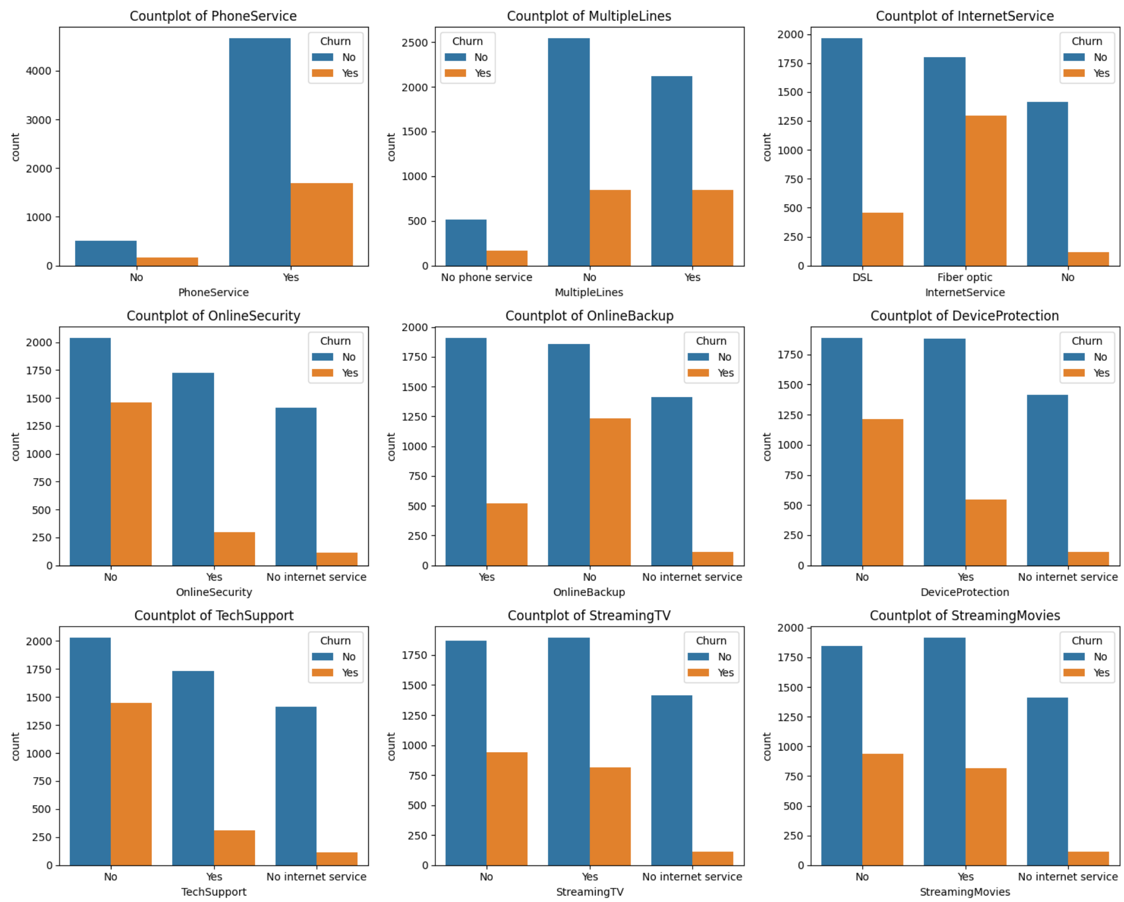
#people who have month to month contract are likely to churn then from who have 1 or 2 years of contract

[77]:
```
df.columns
```

[77]:
```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

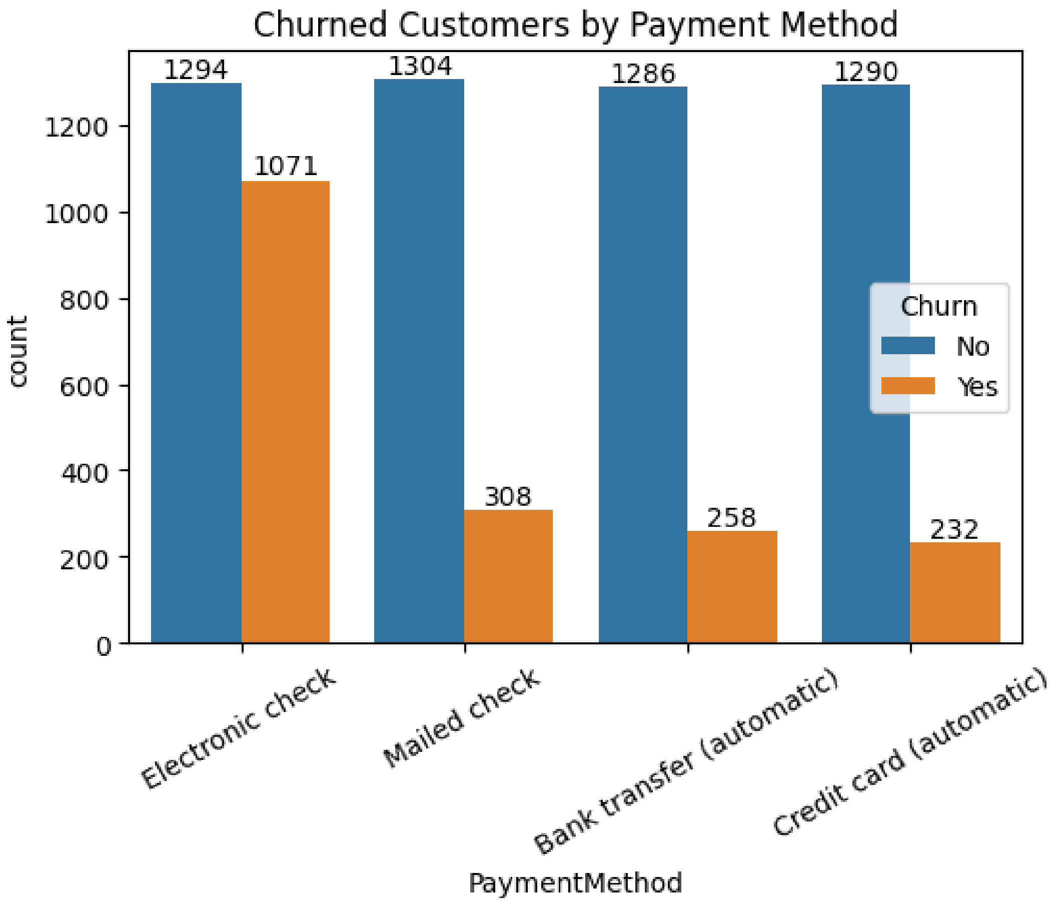
[78]:
```
columns = ['PhoneService', 'MultipleLines', 'InternetService',
           'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
           'TechSupport', 'StreamingTV', 'StreamingMovies']

fig, axes = plt.subplots(3, 3, figsize=(15, 12))
axes = axes.flatten()   # convert into 1d because the loop doesn't work on 2D
```

```
for i, col in enumerate(columns):
    sns.countplot(data=df, x=col, ax=axes[i], hue="Churn")
    axes[i].set_title(f"Countplot of {col}")

plt.tight_layout()   # Auto-adjust subplot spacing
plt.show()
```



#the majority of customers who have not churn tend to have services like PhoneService, InternetService(particulary DSL), and Online Security enabled for service like OnlineBackup, TechSupport, and StreamingTV, churn rates are noticeably higher when these serivces are not used or are unavailable

[79]:
```
plt.figure(figsize=(6,4))
ax = sns.countplot(x="PaymentMethod", data=df, hue="Churn")
ax.bar_label(ax.containers[0])
plt.title("Churned Customers by Payment Method")
plt.xticks(rotation=45)
plt.show()
```



#customer is likely to churn when he is using electronic check as a payment mehtod.